Test Plan for CourseHub

COMP 4350 - Winter 2026

ChangeLog

Version	Change Date	By	Description
1	2nd March 2026	Opeyemi Ogundimu	Initial Testing plan
2	26th March 2026	Opeyemi Ogundimu	Initial mutation testing plan aligned with the Sprint 2 Test Plan
3	2nd April 2026	Opeyemi Ogundimu	Sprint 4 load testing methodology, scenarios, metrics, and execution workflow
4	3rd April 2026	Opeyemi Ogundimu	Added sustained 20-user / 200 RPM acceptance result and mixed baseline plan details
5	3rd April 2026	Opeyemi Ogundimu	Expanded automated test coverage for implemented Sprint 4 features (important dates, dashboard, enrollment)
6	3rd April 2026	Opeyemi Ogundimu	Restructured test plan to align feature numbering and naming with CoreFeatures.md

1. Introduction

1.1 Scope

CourseHub is a full-stack course platform built with Vue.js 3 (frontend), Spring Boot 3 / Java 21 (backend), and PostgreSQL. This test plan covers the six core functional features defined in CoreFeatures.md. Features 4 (Rating & Review System) and 6 (Assignment & Submission) are deferred and out of scope for the current sprint; their tests are not included. All implemented tests for important date management and student dashboard behaviour are covered under Feature 5 (Analytics Dashboards), which governs tracking of upcoming due dates and personalized student progress views.

1.1.1 Functional Scope

Feature 1 - User Authentication & Role Management

- Users can register with name, email, password, and role (Professor or Student) and log in with valid credentials.
- Passwords are hashed before storage; successful authentication establishes a user session and redirects to the role-specific dashboard.
- Role is captured at sign-up and enforces role-specific access behaviour (e.g., restricted professor-only areas).
- Registration and login return clear user-facing errors for duplicate email, invalid credentials, and invalid payload.

Feature 2 - Course Creation & Management

- Professors can create, edit, and delete their own courses; students cannot perform professor-only course management actions.
- Required-field validation, success feedback, redirect behaviour, delete confirmation, and immediate UI state update after deletion.

Feature 3 - Enrollment & Progress Tracking

- Students can browse the course catalog, search by tag, enroll in a course, access enrolled course materials, and remove active enrollments.
- Duplicate enrollment is rejected; unenrolled students cannot access course materials.

Feature 4 - Rating & Review System (Deferred - out of scope)

This feature is not implemented in the current sprint. No tests are included for this feature.

Feature 5 - Analytics Dashboards

- Professors can create, retrieve, update, and delete course-specific important dates, enforcing course ownership and role-based access rules.
- Students access a personalized dashboard that aggregates enrolled-course data and upcoming important dates into a single view.
- Date ordering, blank description normalization, and ownership enforcement are verified.
- Empty-state rendering and content-access revocation after unenrollment are verified.

Feature 6 - Assignment & Submission (Stretch Feature - out of scope)

This is a stretch feature that is not implemented in the current sprint. No tests are included for this feature.

1.1.2 Non-Functional Scope

- Security - Password hashing, authenticated session handling, and role-based authorization checks for all protected actions.
- Data Integrity - Ownership checks on course and important-date updates/deletes and safe handling of dependent records.
- Validation & Correctness - Server and client validation for required fields and consistent HTTP status and error responses.
- Usability - Clear forms, understandable error messages, and predictable redirect behaviour after auth and course actions.
- Reliability - Core workflows are covered by automated tests that must pass CI before merge. Mutation results must be reproducible locally and in CI.
- Test Strength - Mutation testing confirms unit tests detect meaningful code changes rather than only achieving line coverage.
- Fault Detection - Mutation testing is used to expose weak assertions, missing branch coverage, and insufficient negative-path tests.
- Release Readiness - Mutation analysis strengthens confidence in backend services before merge and release.

2. Roles and Responsibilities

Name	Net ID	GitHub Username	Role
Pankaj Jhanji	jhanjip	fruitlesspeak	Full-Stack Developer
Jessie Ttito Tello	ttitotej	jessiettito	Full-Stack Developer
Mohamed Hammam	hammamm	mhammam2	Full-Stack Developer
Opeyemi Ogundimu	ogundimo	ogundimo	Full-Stack Developer

All four members share the Full-Stack Developer role and are collectively responsible for:

- Building and maintaining end-to-end features across the Vue frontend, Spring Boot APIs, and PostgreSQL data models, including auth, validation, and role-based behaviour.
- Ensuring release quality by writing and fixing automated tests, debugging cross-layer issues, and keeping CI/CD checks passing before merge.

3. Test Methodology

3.1 Core Feature 1: User Authentication & Role Management

Secure sign-up and login functionality supporting distinct roles (Professor and Student) to control access to specific features.

3.1.1 Unit Tests

A) Registration Logic (RegisterServiceTest) - 2 tests

- `registerWithValidPayloadSavesUserAndReturnsResponse` - verifies valid registration normalizes input, hashes password, saves user, and returns expected response fields.
- `registerWithExistingEmailThrowsConflictException` - verifies duplicate email throws `EmailAlreadyInUseException` and skips password encoding.

B) Login / Auth Logic (AuthServiceTest) - 6 tests

- `loginWithValidCredentialsCreatesSessionAndReturnsDashboard` - verifies valid login creates session attributes and returns correct role and dashboard path.
- `loginWithUnknownEmailThrowsInvalidCredentials` - verifies unknown email throws `InvalidCredentialsException` and no session is created.
- `loginWithWrongPasswordThrowsInvalidCredentials` - verifies bad password throws `InvalidCredentialsException` and no session is created.
- `getCurrentSessionWhenAuthenticatedReturnsProfile` - verifies existing session returns populated current-session profile.
- `getCurrentSessionWithoutSessionReturnsEmpty` - verifies no session returns empty result.
- `logoutInvalidatesSession` - verifies logout invalidates the active session.

C) Auth & Registration API (AuthControllerTest / RegisterControllerTest) - 8 tests

- `registerWithValidPayloadReturns201AndResponseBody` - POST `/api/auth/register` returns 201 and created user body.
- `registerWithDuplicateEmailReturns409` - duplicate registration returns 409 with conflict message.
- `loginWithValidPayloadReturns200AndResponseBody` - POST `/api/auth/login` returns 200 with userId, role, dashboardPath.
- `loginWithInvalidCredentialsReturns401` - invalid credentials return 401 and error message.
- `loginWithInvalidPayloadReturns422` - invalid payload returns 422 with field errors.
- `sessionWhenAuthenticatedReturns200` - GET `/api/auth/session` returns authenticated session data.
- `sessionWhenUnauthenticatedReturns401` - GET `/api/auth/session` returns 401 when not authenticated.
- `logoutReturns204` - POST `/api/auth/logout` returns 204.

D) Error Handling (ApiExceptionHandlerTest) - 1 test

- `handleEmailConflictReturnsConflictMessage - exception handler returns 409 + "Email already in use" payload.`

3.1.2 Acceptance Tests

Acceptance testing was performed by team members acting as end-users to confirm each user story was met through normal UI usage.

User Story: As a guest user, I want to register with email and password so that I can create a CourseHub account.

- Given I am a guest user on the registration page (/register)
- When I enter a valid full name, email, password, confirmed password, and select a role
- And I click Create Account
- Then the system creates a new account and redirects me to my dashboard (or shows sign-in prompt if auto sign-in is unavailable)

User Story: As a registered user, I want to log in so that I can access my specific dashboard.

- Given I am a registered user on the login page (/login)
- When I enter my registered email and correct password and click Sign In
- Then the system authenticates me and redirects to my dashboard (/dashboard/{userId})

3.1.3 Mutation Tests

A) Registration Logic (RegisterService) - 3 tests

- `mutation_registerWithValidPayloadSavesUserAndReturnsResponse - verifies valid registration normalizes input, hashes password, saves the user, assigns the expected role, and returns the expected response fields.`
- `mutation_registerWithExistingEmailThrowsConflictException - verifies duplicate email registration throws EmailAlreadyInUseException, skips password encoding, and prevents persistence.`
- `mutation_registerWithProfessorRoleSavesProfessorAndReturnsProfessorResponse - verifies professor registration stores the professor role correctly, normalizes the email, hashes the password, and returns the expected professor response data.`

B) Login / Auth Logic (AuthService) - 9 tests

- `mutation_loginWithValidStudentCredentialsCreatesSessionAndReturnsStudentDashboard - verifies valid student login creates the session, stores the correct session attributes, and returns the expected student dashboard path.`
- `mutation_loginWithValidProfessorCredentialsCreatesSessionAndReturnsProfessorDashboard - verifies valid professor login creates the session and returns the expected professor dashboard path.`

- mutation_loginWithUnknownEmailThrowsInvalidCredentials - verifies unknown email login throws InvalidCredentialsException and does not create a session.
- mutation_loginWithWrongPasswordThrowsInvalidCredentials - verifies invalid password login throws InvalidCredentialsException and does not authenticate the user.
- mutation_getCurrentSessionWhenAuthenticatedAsStudentReturnsProfile - verifies an authenticated student session returns the expected current-session profile and dashboard path.
- mutation_getCurrentSessionWhenAuthenticatedAsProfessorReturnsProfessorDashboard - verifies an authenticated professor session returns the expected role-aware dashboard path.
- mutation_getCurrentSessionWithInvalidSessionUserIdReturnsEmptyAndInvalidatesSession - verifies an invalid session user ID is handled safely by invalidating the session and returning an empty result.
- mutation_getCurrentSessionWithoutSessionReturnsEmpty - verifies the service returns an empty result when no HTTP session exists.
- mutation_logoutInvalidatesSession - verifies logout invalidates the active session and kills mutants that skip session invalidation.

3.2 Core Feature 2: Course Creation & Management

A set of tools for professors to create, manage, and delete courses, with role-based access control and ownership enforcement.

3.2.1 Unit Tests

A) Course Service Logic (CourseService) - 10 tests

- `createWithHttpsLinkSavesLinkAsIs - verifies valid https:// link is preserved on create.`
- `createWithWwwLinkNormalizesToHttps - verifies www. link is normalized to https://www. before save.`
- `createWithHttpLinkThrowsValidationError - verifies insecure http:// link is rejected and course is not saved.`
- `createWithMalformedLinkThrowsValidationError - verifies malformed link is rejected and course is not saved.`
- `createWithBlankLinkStoresNull - verifies blank link is normalized to null.`
- `createWithNonProfessorThrowsError - verifies non-professor or invalid professor ID is rejected and no save occurs.`
- `updateCourseAsOwnerUpdatesEditableFieldsAndReturnsResponse - verifies owner can update editable fields and receives the updated response.`
- `updateCourseAsNonOwnerThrowsForbiddenAndDoesNotSave - verifies non-owner professor update is forbidden and no save occurs.`
- `deleteCourseAsOwnerDeletesCourse - verifies owner can delete course.`
- `deleteCourseAsNonOwnerThrowsForbiddenAndDoesNotDelete - verifies non-owner professor delete is forbidden and no delete occurs.`

B) Course API (CourseController) - 12 tests

- `createCourseAsProfessorReturns201AndResponse - POST /api/courses by professor returns 201 with created course.`
- `createCourseAsStudentReturns403 - student create attempt returns 403.`
- `createCourseWithoutSessionReturns401 - unauthenticated create returns 401.`
- `createCourseWithMissingRoleReturns401 - missing session role returns 401.`
- `createCourseWithMissingUserIdReturns401 - missing session user ID returns 401.`
- `createCourseWithBlankTitleReturns422 - blank title fails validation (422).`
- `createCourseWithBlankCodeReturns422 - blank code fails validation (422).`
- `updateAsOwnerProfessorReturns200AndResponseBody - owner professor PATCH /api/courses/{uuid} returns 200 with updated body.`
- `updateAsNonOwnerReturns403 - non-owner professor patch returns 403 with ownership error.`

- `updateAsStudentReturns403WithoutCallingService - student patch returns 403 and service is not called.`
- `deleteAsOwnerProfessorReturns204 - owner professor delete returns 204.`
- `deleteWithoutSessionReturns401WithoutCallingService - unauthenticated delete returns 401 and service is not called.`

3.2.2 Acceptance Tests

Acceptance testing was performed by team members acting as professors to confirm each user story was met through normal UI usage.

User Story: Professor can create a course and it appears in the catalog.

- Given I am logged in as a professor
- When I fill in required course fields and click Create
- Then the course is created, appears in the course catalog, and a student account can view it and enroll

User Story: Professor can edit course details they created.

- Given I am logged in as the professor who created a course and I am viewing that course
- When I click Edit, update description, tags, material, and due date, and save changes
- Then the updated values are persisted and the course displays the new values

User Story: Professor can delete a course they created.

- Given I am logged in as the professor who created a course and I am viewing that course
- When I click Delete and confirm the delete action
- Then the course is permanently removed from the platform and cannot be retrieved by direct course URL or API lookup

3.2.3 Mutation Tests

A) Course Service Logic (CourseService) - 20 tests

- `mutation_createWithHttpsLinkSavesLinkAsIs - verifies a valid https:// course link is preserved exactly during creation.`
- `mutation_createWithWwwLinkNormalizesToHttps - verifies a www. course link is normalized to https:// before save and response mapping.`
- `mutation_createWithHttpLinkThrowsValidationError - verifies insecure http:// links are rejected and the course is not saved.`
- `mutation_createWithMalformedLinkThrowsValidationError - verifies malformed links are rejected and the course is not saved.`
- `mutation_createWithBlankLinkStoresNull - verifies a blank link is normalized to null.`
- `mutation_createWithMissingProfessorThrowsError - verifies course creation fails when the professor does not exist.`

- mutation_createWithExistingNonProfessorUserThrowsError - verifies a non-professor user cannot create a course and the save path is blocked.
- mutation_createWithBlankOptionalTextFieldsStoresNulls - verifies blank optional text fields are normalized to null.
- mutation_createWithNullOptionalFieldsStoresNulls - verifies null optional fields remain null throughout create and response mapping.
- mutation_findAllReturnsMappedCourses - verifies findAll() returns mapped course response objects.
- mutation_findByUuidReturnsMappedCourse - verifies findByUuid() returns the expected mapped course response.
- mutation_findByUuidWhenCourseMissingThrowsNotFound - verifies a missing UUID throws the expected not-found exception.
- mutation_findByProfessorReturnsMappedCourses - verifies professor-specific retrieval returns the correct mapped courses.
- mutation_searchReturnsMappedCourses - verifies search returns the expected filtered and mapped course results.
- mutation_updateCourseAsOwnerUpdatesEditableFieldsAndReturnsResponse - verifies the course owner can update editable fields and receives the correct updated response.
- mutation_updateCourseAsOwnerUpdatesTitleAndCode - verifies the owner can update course title and code successfully.
- mutation_updateCourseAsOwnerBlankOptionalFieldsBecomeNull - verifies blank optional fields are normalized to null during update.
- mutation_updateCourseAsNonOwnerThrowsForbiddenAndDoesNotSave - verifies a non-owner professor cannot update another professor's course.
- mutation_deleteCourseAsOwnerDeletesCourse - verifies the owner can delete the course successfully.
- mutation_deleteCourseAsNonOwnerThrowsForbiddenAndDoesNotDelete - verifies a non-owner professor cannot delete another professor's course.

3.3 Core Feature 3: Enrollment & Progress Tracking

Functionality for students to browse courses via query or tags, enroll, access enrolled course materials, and remove active enrollments.

3.3.1 Unit Tests

A) Enrollment Lifecycle Logic (EnrollmentService) - 14 tests

- `enrollCreatesNewActiveEnrollmentWithStudentAndCourseIds` - verifies a new enrollment stores the correct student ID and course ID and is persisted as active.
- `reactivateInactiveEnrollmentMarksItActiveAndSaves` - verifies an existing inactive enrollment is reactivated and saved rather than duplicated.
- `enrollWhenAlreadyActiveThrowsDuplicateErrorAndDoesNotSave` - verifies duplicate active enrollment attempts throw an error and do not save.
- `deactivateActiveEnrollmentMarksItInactiveAndSaves` - verifies unenrollment deactivates an active enrollment and persists the updated state.
- `deactivateWhenAlreadyInactiveThrowsAndDoesNotSave` - verifies deactivation fails for an already inactive enrollment and does not save.
- `deactivateWhenEnrollmentMissingThrowsNotFound` - verifies deactivation throws the expected not-found exception when no enrollment record exists.
- `enrollWhenStudentMissingThrowsNotFound` - verifies enrollment fails when the student account does not exist.
- `enrollWhenProfessorIdPassedThrowsNoStudentFound` - verifies a professor account cannot be treated as an eligible student enrollment target.
- `enrollWhenCourseMissingThrowsNotFound` - verifies enrollment fails when the target course does not exist.
- `findByUserReturnsActiveEnrollments` - verifies student-specific enrollment lookup returns the expected active enrollments.
- `findByCourseReturnsActiveEnrollments` - verifies course-specific enrollment lookup returns the expected active enrollments.
- `isEnrolledReturnsTrueForActiveEnrollment` - verifies active enrollment status is reported as true.
- `isEnrolledReturnsFalseForInactiveEnrollment` - verifies inactive enrollments are not treated as active access.
- `isEnrolledReturnsFalseWhenEnrollmentMissing` - verifies missing enrollment records return false.

B) Enrollment API (CourseController) - 5 tests

- `enrollAsStudentReturns201AndCallsService` - verifies an authenticated student can enroll and the enrollment service is called with the expected identifiers.
- `getMyCoursesAsStudentReturnsCourses` - verifies an authenticated student can retrieve enrolled courses through `GET /api/courses/my-courses`.

- `listWithTagReturnsFilteredCourses - verifies controller-level tag filtering returns the expected filtered course list.`
- `getCourseMaterialAsEnrolledStudentReturns200 - verifies an enrolled student can access course material through GET /api/courses/{uuid}/content.`
- `getCourseMaterialAsNonEnrolledStudentReturns403 - verifies an unenrolled student is rejected from the content endpoint.`

3.3.2 Acceptance Tests

Acceptance testing was performed by end-users and team members acting as student users to confirm enrollment and progress tracking requirements were met through normal UI usage.

User Story: Student can browse available courses in the catalog.

- Given I am logged in as a student and navigate to the course catalog
- Then I can see a list of available courses, each showing basic information and an option to enroll if not already enrolled

User Story: Student can search for courses by tag.

- Given I am logged in as a student and viewing the course catalog
- When I enter a tag (e.g., Java) in the search/filter field and submit
- Then only courses containing that tag are displayed

User Story: Student can enroll in a course from the catalog.

- Given I am logged in as a student and viewing the course catalog
- When I click Enroll on a course I am not yet enrolled in
- Then the enrollment completes successfully, I am added to the student list, and the course is marked as enrolled in the UI

User Story: Student can view enrolled courses in My Courses.

- Given I am logged in as a student and enrolled in one or more courses
- When I navigate to the My Courses page
- Then I can see all courses I am currently enrolled in

User Story: Student gains access to course materials after enrolling.

- Given I am logged in as a student and have enrolled in a course
- When I open that course's detail or materials page
- Then I am allowed to access the course materials and the content is no longer locked

User Story: Student cannot enroll in the same course twice.

- Given I am logged in as a student and already enrolled in a course
- When I attempt to enroll in that same course again
- Then the system prevents a duplicate enrollment and I see a clear message such as "Already enrolled"

User Story: Non-enrolled student cannot access course materials.

- Given I am logged in as a student but not enrolled in a course
- When I try to open that course's materials
- Then access is denied and I see a message indicating enrollment is required

3.3.3 Mutation Tests

A) Enrollment Lifecycle Logic (EnrollmentServiceTest) - 14 tests

- `mutation_enrollCreatesNewActiveEnrollmentWithStudentAndCourseIds` - verifies a new enrollment stores the correct student ID and course ID and is persisted as active.
- `mutation_reactivateInactiveEnrollmentMarksItActiveAndSaves` - verifies an existing inactive enrollment is reactivated and saved rather than duplicated.
- `mutation_enrollWhenAlreadyActiveThrowsDuplicateErrorAndDoesNotSave` - verifies duplicate active enrollment attempts throw an error and do not save.
- `mutation_deactivateActiveEnrollmentMarksItInactiveAndSaves` - verifies unenrollment deactivates an active enrollment and persists the updated state.
- `mutation_deactivateWhenAlreadyInactiveThrowsAndDoesNotSave` - verifies deactivation fails for an already inactive enrollment and does not save.
- `mutation_deactivateWhenEnrollmentMissingThrowsNotFound` - verifies deactivation throws the expected not-found exception when no enrollment record exists.
- `mutation_enrollWhenStudentMissingThrowsNotFound` - verifies enrollment fails when the student account does not exist.
- `mutation_enrollWhenProfessorIdPassedThrowsNoStudentFound` - verifies a professor account cannot be treated as an eligible student enrollment target.
- `mutation_enrollWhenCourseMissingThrowsNotFound` - verifies enrollment fails when the target course does not exist.
- `mutation_findByUserReturnsActiveEnrollments` - verifies student-specific enrollment lookup returns the expected active enrollments.
- `mutation_findByCourseReturnsActiveEnrollments` - verifies course-specific enrollment lookup returns the expected active enrollments.
- `mutation_isEnrolledReturnsTrueForActiveEnrollment` - verifies active enrollment status is reported as true.
- `mutation_isEnrolledReturnsFalseForInactiveEnrollment` - verifies inactive enrollments are not treated as active access.
- `mutation_isEnrolledReturnsFalseWhenEnrollmentMissing` - verifies missing enrollment records return false.

B) Enrollment and Discovery Logic (CourseServiceTest) - 4 tests

- `mutation_findByTagReturnsMatchingCourses` - verifies tag-based course filtering returns only the expected matching courses.

- mutation_findMyCoursesReturnsActiveEnrolledCourses - verifies active enrollments are mapped into the correct enrolled-course response list.
- mutation_ensureStudentEnrolledAllowsActiveEnrollment - verifies enrolled students pass the course-access guard.
- mutation_ensureStudentEnrolledRejectsMissingEnrollment - verifies unenrolled students are blocked by the course-access guard with the expected exception.

3.4 Core Feature 4: Rating & Review System (Deferred)

This feature is defined in CoreFeatures.md as a feedback system allowing students to rate courses and write reviews, which are aggregated and displayed on course detail pages.

This feature has not been implemented in the current sprint. No unit tests, integration tests, acceptance tests, mutation tests, or load test scenarios are included for this feature. It will be addressed in a future sprint.

3.5 Core Feature 5: Analytics Dashboards

Personalized dashboards for students to track upcoming due dates and progress. This includes course-specific important date management (used to populate upcoming due dates) and the student dashboard view that aggregates enrolled-course data, important dates, and enrolled-content access.

3.5.1 Unit Tests - Important Date Management (ImportantDateService / ImportantDateController)

A) Important Date Service Logic (ImportantDateService) - 7 tests

- `createForOwnedCourseSavesNormalizedDescription - verifies important-date creation stores the correct course and professor identifiers and normalizes the description field.`
- `createForNonOwnerThrowsForbiddenAndDoesNotSave - verifies a non-owner professor cannot create an important date for another professor's course.`
- `createWithNonProfessorThrowsError - verifies important-date creation fails when the acting user is not a professor.`
- `findByCourseThrowsWhenCourseMissing - verifies course-level important-date retrieval fails with the expected not-found contract when the course does not exist.`
- `findByCourseInDateRangeWithEmptyCourseIdsReturnsEmptyList - verifies date-range retrieval returns an empty result when no course IDs are provided.`
- `updateOwnedImportantDateAppliesProvidedFields - verifies the course owner can update important-date fields and normalize blank descriptions.`
- `deleteAsNonOwnerThrowsForbiddenAndDoesNotDelete - verifies a non-owner professor cannot delete an important date.`

B) Important Date API (ImportantDateController) - 5 tests

- `createAsProfessorReturns201 - verifies the controller returns 201 for professor important-date creation.`
- `createWithoutSessionReturns401 - verifies the controller rejects unauthenticated important-date creation requests.`
- `createAsStudentReturns403 - verifies the controller rejects student important-date creation requests.`
- `updateWithInvalidRoleInSessionReturns401 - verifies invalid session role state is rejected during important-date updates.`
- `deleteAsProfessorReturns204 - verifies the controller returns 204 for professor important-date deletion.`

3.5.2 Unit Tests - Student Dashboard (Frontend)

A) Important Date Store (importantDateStore.spec.ts) - 7 tests

- `fetchByCourse populates important dates and resets loading - verifies the important-date store loads course dates and clears the loading flag correctly.`

- fetchByCourses aggregates important dates across courses - verifies the store combines results from multiple enrolled courses.
- fetchByCourses clears state immediately when no course ids are provided - verifies the store returns empty dashboard-date state when the student has no enrolled courses.
- fetchByCourse stores an api error message and clears loading on failure - verifies frontend error handling for important-date retrieval failures.
- create appends a new important date to the store - verifies store state is updated correctly after important-date creation.
- update replaces an existing important date in the store - verifies store state is updated correctly after important-date modification.
- remove deletes the important date from the store - verifies store state is updated correctly after important-date deletion.

B) Student Dashboard View (StudentDashboardView.spec.ts) - 4 tests

- loads enrolled courses and important dates on mount and renders summary counts - verifies the dashboard loads initial data and displays correct summary values.
- renders the empty state when the student has no enrolled courses - verifies the dashboard renders the correct empty-state messaging.
- drops a course and refreshes the dashboard state - verifies the dashboard refreshes enrolled-course and important-date state after a course is dropped.
- routes to course detail with the dashboard query flag - verifies dashboard course navigation preserves the expected route context.

3.5.3 Acceptance Tests

Acceptance testing for the Analytics Dashboards feature was performed by team members acting as professors and student users to confirm that important-date management and student dashboard requirements were met through normal UI and API usage.

User Story: Professor can manage important dates for a course they own.

- Given I am logged in as a professor who owns a course
- When I create an important date with a title and due date for that course
- Then the important date is saved and appears in the course's date list in chronological order
- And I can update or delete it, with ownership enforced (other professors cannot modify it)

User Story: Student dashboard shows upcoming important dates for enrolled courses.

- Given I am logged in as a student enrolled in one or more courses
- When I navigate to my dashboard
- Then I see a summary of enrolled courses and their upcoming important dates
- And if I have no enrolled courses, an appropriate empty state is displayed

3.5.4 Mutation Tests

A) Important Date Service Logic (ImportantDateService) - 8 tests

- mutation_createWithBlankDescriptionStoresNull - verifies a blank important-date description is normalized to null during creation.
- mutation_createWhenCourseMissingThrowsNotFound - verifies important-date creation fails with the expected not-found error when the target course does not exist.
- mutation_createWhenProfessorMissingThrowsError - verifies important-date creation fails when the acting professor record does not exist.
- mutation_findByCourseReturnsDatesOrderedByDueAt - verifies course-specific important dates are returned in ascending due-date order.
- mutation_findByIdReturnsMappedImportantDate - verifies findById() returns the expected mapped important-date response.
- mutation_findByIdWhenMissingThrowsNotFound - verifies findById() throws the expected not-found exception for a missing important date.
- mutation_findByCourseInDateRangeReturnsOnlyMatchingDates - verifies date-range filtering returns only the important dates within the requested interval.
- mutation_deleteAsOwnerDeletesImportantDate - verifies the course owner can delete an important date successfully.

3.6 Core Feature 6: Assignment & Submission (Stretch Feature)

This feature is defined in CoreFeatures.md as a system where professors can post assignments and students can upload submissions for review.

This is a stretch feature that has not been implemented in the current sprint. No unit tests, integration tests, acceptance tests, mutation tests, or load test scenarios are included. It will be addressed if time permits in a future sprint.

3.7 Integration Tests

Integration tests verify multi-step authenticated workflows spanning multiple API endpoints, database persistence, authorization contracts, and cross-feature interactions. Total: 19 tests.

3.7.1 Authentication and Session Workflows (Features 1 & 2)

1. Register professor + student test - create one professor and one student via API and confirm both persist with correct role flags.
2. Duplicate registration conflict test - attempt registering an existing email and verify HTTP 409 with the expected conflict message.
3. Login/session integration test - login as each role and confirm GET /api/auth/session returns correct role, userId, and dashboard path.
4. Logout/session invalidation test - logout, then call GET /api/auth/session and verify 401.
5. Logout invalidation protection test - login, perform logout, then attempt a protected action with the same session and confirm subsequent protected call returns 401.

3.7.2 Course Management Workflows (Feature 2)

6. Professor course creation success test - create a course as professor and confirm API returns 201, course appears in professor dashboard, and row exists in courses table.
7. Course creation validation test - submit blank title or code and verify 422 with no DB insert.
8. Role authorization test (create) - attempt POST /api/courses as student and verify 403.
9. Ownership authorization test (update/delete) - as a different professor, attempt PATCH and DELETE on another professor's course and verify 403.
10. Owner edit integration test - as the owning professor, update description, tags, material, dueDate, and link, and verify API response, dashboard, and DB all reflect new values.
11. Owner delete + cascade test - delete a course as owner and verify API returns 204, course disappears from dashboard, and related enrollment rows are removed.
12. Validation safety invariant test - submit invalid course creation and confirm API returns 422 and total course count remains unchanged.
13. Multi-user data isolation test - create a course as Professor A, then login as Professor B and confirm Professor B cannot see Professor A's courses.
14. Course search integration pipeline test - create multiple courses and verify search endpoint returns 200, filters by title keyword, and excludes non-matching courses.

3.7.3 Enrollment, Analytics Dashboard, and Progress Workflows (Features 3 & 5) - Automated (SystemIntegrationTest)

15. studentCanDiscoverEnrollAccessContentAndUnenroll - verifies a professor-created course can be discovered by a student, enrolled in, retrieved through GET /api/courses/my-courses, accessed through GET /api/courses/{uuid}/content, and then properly blocked after unenrollment.
16. studentOnlyEnrollmentEndpointsRejectProfessorAndAnonymousUsers - verifies professor and unauthenticated requests to student-only enrollment endpoints are rejected with the expected authorization contract.

17. ownerProfessorCanManageImportantDatesThroughCrudFlow - verifies the course owner can create, list, fetch by ID, update, and delete important dates through the backend API.
18. importantDateAuthorizationAndNotFoundContractsHold - verifies non-owner professor access is rejected, student mutation attempts are rejected, and missing-course or missing-important-date requests return the expected 404 contract.
19. studentDashboardApiFlowReflectsEnrollmentAndImportantDates - verifies the dashboard-dependent API sequence returns enrolled courses, related important dates, and enrolled course content, and then revokes content access after unenrollment.

3.8 Platform / Smoke Test

BackendApplicationTests - 1 test

- `contextLoads – verifies the Spring application context boots successfully with the test configuration.`

3.9 CI/CD Regression Workflow

Regression testing is automated through three GitHub Actions pipelines:

Backend CI (.github/workflows/ci.yml)

- Trigger: push to main or pull request targeting main
- Environment: ubuntu-latest with PostgreSQL 15 service container
- Steps: compile → mvn test → JaCoCo coverage report → upload test artifacts

Frontend CI (.github/workflows/frontend-ci.yml)

- Trigger: push to main or pull request targeting main
- Steps: npm ci → lint → type-check → npm run test:unit -- --run → build

Mutation Testing CI (.github/workflows/mutation.yml)

- Trigger: pull requests or push to main on backend paths; also available for manual dispatch
- PIT targets: AuthService, RegisterService, CourseService, EnrollmentService, ImportantDateService
- Outputs: HTML + XML mutation reports uploaded as artifacts (7-day retention)

Every push or pull request triggers the unit and integration suites. Merging to main is blocked unless all tests pass.

3.10 Load Testing

Load testing is conducted at the backend API level using Apache JMeter 5.6+ against the CourseHub Spring Boot application (port 8080). The goal is to verify that all implemented features can handle realistic concurrent usage without unacceptable response-time degradation, session instability, or database errors. Features 4 (Rating & Review) and 6 (Assignment & Submission) are excluded as they are not implemented.

Feature	Request Type 1 (Read / GET)	Request Type 2 (Write / POST)
Feature 1: User Authentication & Session Management	GET /api/auth/session	POST /api/auth/register POST /api/auth/login POST /api/auth/logout
Feature 2: Course Creation & Management	GET /api/courses?professorId={id} GET /api/courses/{uuid}	POST /api/courses PATCH /api/courses/{uuid} DELETE /api/courses/{uuid}
Feature 3: Enrollment & Progress Tracking	GET /api/courses GET /api/courses?tag={tag} GET /api/courses/my-courses GET /api/courses/{uuid}	POST /api/courses/{uuid}/enroll DELETE /api/courses/{uuid}/enroll
Feature 5: Analytics Dashboards (Important Dates)	GET /api/important-dates?courseId={courseId} GET /api/important-dates/{id}	POST /api/courses/{courseId}/important-dates PATCH /api/important-dates/{id} DELETE /api/important-dates/{id}
Feature 5: Analytics Dashboards (Student Dashboard)	GET /api/courses/my-courses GET /api/important-dates?courseId={courseId} GET /api/courses/{uuid}/content	Indirect write dependency through enrollment and professor date updates; no dedicated dashboard POST endpoint

3.10.1 Load Test Objectives

- Establish a repeatable performance baseline for the implemented backend APIs.
- Verify that read-heavy student traffic and write-heavy professor traffic both remain stable under concurrent access.
- Measure response times, throughput, and error rate for the main user journeys.
- Detect session-handling issues, data-contention problems, and slow database-backed endpoints.

3.10.2 Workload Model

- 65% student traffic - browsing, enrollment, and dashboard reads
- 25% professor traffic - course and important-date management writes
- 10% guest/authentication traffic - short login and registration bursts

Acceptance-level run: 2 authentication users at 20 RPM + 5 professor users at 50 RPM + 13 student users at 130 RPM = 20 virtual users, 200 RPM total.

3.10.3 Load Test Scenarios

20. Authentication Burst (auth.jmx) - POST /api/auth/register, POST /api/auth/login, GET /api/auth/session, POST /api/auth/logout under short bursts of concurrent auth traffic.
21. Professor Management Workflow (course-management.jmx) - POST /api/courses, GET /api/courses?professorId={id}, PATCH /api/courses/{uuid}, DELETE /api/courses/{uuid}, POST /api/courses/{courseId}/important-dates, GET /api/important-dates?courseId={courseId} during repeated professor CRUD activity.
22. Student Discovery and Enrollment Peak (discovery-enrollment.jmx) - GET /api/courses, GET /api/courses?tag={tag}, GET /api/courses/{uuid}, POST /api/courses/{uuid}/enroll, DELETE /api/courses/{uuid}/enroll, GET /api/courses/my-courses during a concentrated enrollment window.
23. Dashboard Fan-Out Read Load - the multi-request student dashboard pattern combining GET /api/courses/my-courses, repeated GET /api/important-dates?courseId={courseId}, GET /api/courses/{uuid}, and GET /api/courses/{uuid}/content.
24. Endurance Run - lower-concurrency sustained traffic to detect memory growth, session instability, or gradually increasing database latency.

3.10.4 Success Metrics

- Sustain at least 20 virtual users generating 200 requests per minute for a minimum of 60 seconds
- 0% unexpected 5xx responses
- Less than 2% unexpected request failures for a completed scenario
- P95 under 2 seconds for read-heavy API requests
- P95 under 3 seconds for write-heavy API requests

3.10.5 Current Acceptance Result

Using mixed-baseline-200rpm.jmx, CourseHub successfully sustained 20 concurrent virtual users, 200 HTTP requests over 59.9 seconds, at 200.33 RPM, with 0% recorded request failures. This result was measured against a local Spring Boot backend with a temporary H2 configuration. The suite should be re-run against the Docker/PostgreSQL environment for the production-representative baseline.

3.10.6 Load Testing Execution Workflow

25. Run unit and integration tests on every push and pull request as the primary regression gate.
26. Run the JMeter load suites manually for milestone reviews, before major merges to main, and after backend changes that could affect performance.
27. Store generated .jtl files and HTML dashboard reports outside version control.
28. Record important baseline numbers and regressions in project documentation.
29. Re-run the affected scenario after any performance fix to confirm the regression is resolved.

4. Test Level Summary

Test Level	Scope & Requirement	Methodology
Unit Testing	77 backend unit tests + 11 frontend unit/component tests = 88 total. Feature 1: 17 tests (8 controller + 6 auth service + 2 register service + 1 error handler) Feature 2: 22 tests (10 service + 12 controller) Feature 3: 19 tests (14 service + 5 controller) Feature 5: 12 backend + 11 frontend = 23 tests	Backend: JUnit 5, Mockito, MockMvc, and Spring Boot Test. Frontend: Vitest, Vue Test Utils, and Pinia-backed store tests.
Integration Testing	19 tests total covering multi-step authenticated API workflows across all implemented features.	Spring Boot integration tests with MockMvc and real repository-backed assertions. Tests execute multi-step authenticated workflows verifying response contracts, persistence, and authorization.
Acceptance Testing	End-user testing for every user story in Features 1, 2, 3, and 5.	Team members perform manual walkthroughs based on user story criteria, verifying each acceptance condition in the running application.
Regression Testing	Unit + Integration tests executed on every push/PR to the main branch.	GitHub Actions CI pipeline (ci.yml + frontend-ci.yml) runs all backend and frontend tests automatically. Merging to main is blocked unless all tests pass.
Mutation Testing	58 mutation-specific test methods across 5 backend service classes. Feature 1: 12 (RegisterService 3 + AuthService 9) Feature 2: 20 (CourseService) Feature 3: 18 (EnrollmentService 14 + CourseService 4) Feature 5: 8 (ImportantDateService) PIT result: 123/125 mutants killed Mutation score: 98% Test strength: 99%	PIT (Pitest Maven Plugin v1.19.4) targets RegisterService, AuthService, CourseService, EnrollmentService, and ImportantDateService. PIT generates code mutants, re-runs the JUnit 5 test suite, and reports which mutants were killed or survived.
Load Testing	API-level concurrent testing for all implemented features. Baseline: 20 VUs, 200 RPM, 60+ seconds sustained. Result: 200 requests / 59.9 s at 200.33 RPM, 0% failures.	Apache JMeter simulates guest, student, and professor traffic. Four JMeter plans: auth.jmx, course-management.jmx, discovery-enrollment.jmx, mixed-baseline-200rpm.jmx. Load tests run manually, not in CI.

5. Terms and Acronyms

Term / Acronym	Definition
API	Application Program Interface
AUT	Application Under Test
CI/CD	Continuous Integration / Continuous Delivery
PIT	Mutation testing tool for Java that generates code mutants and evaluates whether the test suite detects them
Mutant	A modified version of the original program created to simulate a small fault
Killed Mutant	A mutant detected by one or more failing tests
Survived Mutant	A mutant that does not cause any related test to fail
Equivalent Mutant	A mutant that changes the code syntactically but not the observable behavior
MockMvc	Spring's test framework for executing HTTP requests against controller endpoints in automated tests without deploying the application
Vitest	The frontend unit and component testing framework used for Vue and Pinia test execution in this repository
Mutation Score	The percentage of generated mutants that are killed during a PIT run
Test Strength	In mutation testing, the percentage of covered mutants that are killed by the test suite
JMeter	Apache JMeter, the load testing tool used to generate concurrent API traffic and performance reports
Virtual User (VU)	A simulated user in a load test scenario
Throughput	The rate at which requests are successfully processed during a test run
P95	The 95th percentile response time; 95% of requests complete at or below this value
Ramp-Up	The time period during which virtual users are gradually added to a test